SGD with Momentum

(Optimizers in Deep Learning – Part 2)

---

1. Why We Need Momentum

Before Momentum, we used **Stochastic Gradient Descent (SGD)**.
In SGD, the model updates weights step by step, moving opposite to the gradient to reduce loss.
But there's a problem

➢ Problem — Zig-Zag Path

When the loss surface is shaped like a **valley**, gradients can be steep in one direction (vertical) and shallow in another (horizontal).
SGD keeps jumping **left-right** across the valley walls — wasting time — instead of moving smoothly toward the bottom (the minimum).
That's why learning looks *zig-zag* and **slow**.

---

2. Intuition Behind Momentum

Think of **Momentum** like a **ball rolling down a hill**

- If the ball keeps rolling in the same direction, it gains **speed** (momentum).
- If the slope changes, the momentum helps it **smoothly continue forward** instead of getting stuck.

Similarly, in optimization:

- Momentum helps the model **remember** the past gradients.
- It **reduces oscillations** and **accelerates** learning in the right direction.

So, instead of bouncing around, the model **glides smoothly** toward the minimum.

---

3. How Momentum Works

Momentum adds an extra term to the weight update rule — a "velocity" term that remembers previous updates.
Let's define:

Symbol	Meaning
θ	Model parameters (weights)
v_t	Velocity (moving average of gradients)
β	Momentum coefficient ($0 \leq \beta < 1$)

η	Learning rate
$\nabla_\theta J(\theta_t)$	Gradient at time *t*

Step 1: Compute the Velocity

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta_t)$$

- This is **Exponentially Weighted Moving Average (EWMA)** of gradients.
- It smooths out fluctuations by averaging the direction of past gradients.

Step 2: Update the Weights

$$\theta_t = \theta_{t-1} - \eta v_t$$

Now, instead of updating directly with the current gradient,
the model updates using the **smoothed gradient** (the momentum).

---

4. The Role of β (Beta)

The value of **β** controls how much past gradients influence the current update:

β value	Effect
0	No momentum → Normal SGD
0.5	Moderate smoothing
0.9	Strong momentum → smoother, faster convergence

Common choice: **β = 0.9**

If β is too high → the optimizer may "overshoot" (move too fast and miss the minimum).

If too low → not enough smoothing, still zig-zag.

---

5. Momentum's Effect (Visual Intuition)

Imagine the loss surface as a **bowl**.

Without Momentum:

- The ball bounces left-right (zig-zag).
- Slow to reach the bottom.

With Momentum:

- The ball gains speed downhill.
- Moves mostly forward (horizontally) while oscillations reduce.
- Reaches the bottom faster.

That's why Momentum is like **adding inertia** — it smooths the trajectory.

---

6. Convex vs Non-Convex Surfaces

- In **convex** surfaces (like a bowl) → only one global minimum; momentum helps reach it faster.
- In **non-convex** surfaces (complex, multi-valley) → momentum helps escape small local minima and move toward deeper valleys.

---

7. Mathematical Summary

Here's the Momentum update formula (in one place):

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta_t)$$
$$\theta_t = \theta_{t-1} - \eta v_t$$

Alternate (simplified) version used often:

$$v_t = \beta v_{t-1} + \nabla_\theta J(\theta_t)$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

(Here we skip (1−β) since it's a scaling factor.)

---

9. Problems with Momentum

While Momentum helps speed up convergence, it's not perfect:

1. **Overshooting**
    - If β is too high, it may skip over the minimum due to excessive speed.
2. **Slower to stabilize near minima**
    - Because the momentum keeps the optimizer moving, it can take time to "settle down" once close to the bottom.
3. **Requires tuning of β**
    - β must be chosen carefully (commonly 0.9 or 0.99).

---

10. Why Momentum Works So Well

- Reduces oscillations
- Speeds up convergence
- Helps escape small local minima
- Uses EWMA → smoother updates
- Works great with SGD on noisy data

---

11. Relation to EWMA (Connection)

Remember EWMA from the previous lesson?

Momentum = **SGD + EWMA of gradients**

$$v_t = \text{EWMA of gradients over time}$$

That's why momentum "remembers" the direction of movement — instead of reacting only to the current batch's noise.

---

12. Summary Table

Concept	Description
Goal	Reduce zig-zag and accelerate convergence
Uses	Exponentially Weighted Average of Gradients
Parameter	β (momentum coefficient)
Effect	Smooths trajectory, stabilizes learning
Drawback	May overshoot if β too high
Typical β	0.9

📎 13. Key Intuition Recap

Analogy	Meaning
Rolling ball down hill	Learner keeps its speed (momentum)
EWMA smoothing	Past gradients fade slowly
Zig-zag to straight	Faster, smoother training path
With momentum	Faster learning and better direction

➢ TL;DR (In One Line)

Momentum helps the model learn faster by remembering the direction of past gradients — smoothing out noisy updates and reducing zig-zag motion.
